Punto di Avvio (Package: `it.unipv.ingsfw.playfieldtech.app`)

Questo è il file che avvia l'intera applicazione.

1. `App.java`

- **Ruolo Architetturale:** Entry Point (Punto di Ingresso).
- **Descrizione Dettagliata:** Questa classe contiene l'unico metodo `public static void main(String[] args)`, che è il punto di partenza obbligatorio per qualsiasi applicazione Java.
- **Logica Fondamentale:**
    1. `SwingUtilities.invokeLater(...)`: Questo comando è fondamentale in Swing. Ordina a Java di creare l'interfaccia grafica (GUI) in un thread dedicato (l'Event Dispatch Thread o EDT). Questo previene errori grafici e assicura che l'applicazione sia reattiva.
    2. `MainFrame frame = new MainFrame();`: Crea l'oggetto **Vista** principale (la finestra).
    3. `new AppController(frame);`: Crea l'oggetto **Controller** (il "cervello") e gli passa un riferimento diretto alla finestra (`frame`) che deve controllare.
    4. `frame.showLoginView();`: Dice alla finestra di mostrare il pannello di Login come prima schermata.
    5. `frame.setVisible(true);`: Rende la finestra visibile all'utente.

🧠 Il Controller (Package: `it.unipv.ingsfw.playfieldtech.controller`)

Questo è il "cervello" dell'applicazione. Collega la Vista (i pulsanti) alla logica e ai dati.

2. `AppController.java`

- **Ruolo Architetturale:** Controller (nel pattern MVC).
- **Descrizione Dettagliata:** Questa è la classe più importante. È il "regista" che orchestra l'intera applicazione. Non contiene componenti grafici, ma solo logica.
- **Logica Fondamentale:**
    - **Campi:** Contiene un riferimento a `MainFrame` (la Vista) e a *tutti i DAO* (per l'accesso ai dati). Mantiene anche lo "stato" dell'app, come `utenteCorrente` (chi è loggato) e `dataVisualizzata` (quale giorno del calendario si sta guardando).

- **`initController()` (Metodo Chiave):** Questo metodo, chiamato nel costruttore, è il cuore del controller. Collega i componenti della Vista alla logica del Controller. Esegue `addActionListener` su *ogni* pulsante. Ad esempio:
    1. `loginView.addLoginListener(_ -> eseguiLogin());` dice: "Quando il pulsante di login viene premuto, esegui il *mio* metodo `eseguiLogin()`".
- **`eseguiLogin()`**:
    1. Prende username e password dalla `LoginView`.
    2. Chiama `utenteDAO.validaLogin()`.
    3. Se il risultato è `null`, mostra un pop-up di errore (`JOptionPane`).
    4. Se il risultato è valido, controlla (`instanceof`) se è `Cliente` o `Amministratore`.
    5. Chiama `mainFrame.showClienteView()` o `mainFrame.showAdminView()` per cambiare pannello.
- **`aggiornaCalendarioView()`**:
    1. Un metodo complesso che costruisce la griglia del calendario.
    2. Chiama *quattro* DAO diversi per ottenere tutti i dati necessari: `impiantoDAO`, `fasciaOrariaDAO`, `prenotazioneDAO` (per la data odierna) e `impiantiFasceDAO` (per la mappa delle disponibilità).
    3. Filtra le fasce orarie per il giorno della settimana selezionato.
    4. Popola un `DefaultTableModel` cella per cella. Per ogni incrocio Impianto/Fascia, controlla: (1) L'impianto è operativo in quella fascia? (2) Se sì, è già occupato?
    5. Scrive "Libero", "OCCUPATO" o "" (vuoto) nella cella.
    6. Imposta il modello sulla `calendarioTable` della `ClienteView`.
- **`eseguiPrenotazione()`**:
    1. Controlla quale cella della tabella è selezionata.
    2. Verifica che la cella sia "Libero".
    3. Apre il `PrenotazioneDialog`, passandogli il prezzo (preso dalla `fasciaSelezionata`).
    4. Se l'utente conferma, valida i 10 username dei giocatori (chiamando `utenteDAO.findUtenteByUsername` in un ciclo).
    5. Applica la **regola di business**: controlla che l'ID dell'`utenteCorrente` sia presente nella lista dei 10 giocatori.
    6. Simula un pagamento con `JOptionPane.showConfirmDialog`.
    7. Se tutto è OK, crea l'oggetto `Prenotazione` e lo passa al `prenotazioneManager` per il salvataggio.
- **Altri Metodi:** Gestiscono le altre azioni dei pulsanti (logout, registrazione, cambio giorno, apertura dialoghi admin).

🖥️ La Vista (Package: `it.unipv.ingsfw.playfieldtech.view`)

Questi file definiscono l'Interfaccia Grafica (GUI). Sono "stupidi": mostrano solo dati e dicono al Controller quando l'utente fa qualcosa.

3. `MainFrame.java`

- **Ruolo Architetturale:** View (Finestra Principale).
- **Classe:** `JFrame`.
- **Descrizione Dettagliata:** È la finestra principale che contiene tutto. Il suo componente chiave è un `JPanel` con un **`CardLayout`**. Questo layout speciale agisce come un mazzo di carte, permettendo di mostrare un solo pannello alla volta (es. `LoginView`, `ClienteView`).
- **Metodi Chiave:** `showLoginView()`, `showClienteView()`, ecc. Questi metodi sono chiamati dal `AppController` per cambiare la schermata visibile.

4. `LoginView.java`

- **Ruolo Architetturale:** View (Pannello).
- **Classe:** `JPanel`.
- **Descrizione Dettagliata:** È la schermata di login. Contiene `JTextField` per username e `JPasswordField` per la password.
- **Metodi Chiave:** `getUsername()`, `getPassword()` (usati dal Controller per leggere i dati), `addLoginListener()` (usato dal Controller per attaccare la logica al pulsante).

5. `RegistrazioneView.java`

- **Ruolo Architetturale:** View (Pannello).
- **Classe:** `JPanel`.
- **Descrizione Dettagliata:** Simile a `LoginView`, è un modulo (`form`) con campi di testo (`nomeField`, `cognomeField`, ecc.) per la registrazione di un nuovo utente.
- **Metodi Chiave:** `getNome()`, `getPassword()`, `addRegistrazioneListener()`, ecc.

6. `ClienteView.java`

- **Ruolo Architetturale:** View (Pannello) e Observer.
- **Classe:** `JPanel` che implementa `Observer`.
- **Descrizione Dettagliata:** È la dashboard del cliente. Il suo componente principale è la `JTable calendarioTable`. Contiene anche i pulsanti `prenotaButton`,

`storicoButton` e i pulsanti per navigare i giorni (`giornoPrecedenteButton`, `giornoSuccessivoButton`).

- **Logica (Observer):** Implementa il metodo `update()` dall'interfaccia `Observer`. Quando viene chiamato (dal `PrenotazioneManager`), mostra un pop-up di conferma "Prenotazione avvenuta!". L'aggiornamento vero e proprio della tabella viene poi gestito dal `AppController`.

7. `AdminView.java`

- **Ruolo Architetturale:** View (Pannello).
- **Classe:** `JPanel`.
- **Descrizione Dettagliata:** La dashboard dell'amministratore. Contiene i pulsanti per le funzioni admin: `aggiungiImpiantoButton` e `visualizzaPrenotazioniButton`.

8. `AggiungiImpiantoDialog.java`

- **Ruolo Architetturale:** View (Finestra Pop-up).
- **Classe:** `JDialog` (una finestra "modale", che blocca quella principale).
- **Descrizione Dettagliata:** Viene aperta dall'`AppController` quando l'admin clicca "Aggiungi Impianto". Il costruttore riceve la `List<FasciaOraria>` (letta dal DAO) e crea dinamicamente una `JCheckBox` per ogni fascia.
- **Metodi Chiave:** `getNomeImpianto()` e `getSelectedFasce()`. Quest'ultimo scorre tutte le checkbox e restituisce una lista degli oggetti `FasciaOraria` selezionati.

9. `PrenotazioneDialog.java`

- **Ruolo Architetturale:** View (Finestra Pop-up).
- **Classe:** `JDialog`.
- **Descrizione Dettagliata:** Viene aperta dall'`AppController` durante una prenotazione. Il costruttore riceve il `prezzo` (calcolato dal Controller) e lo mostra in una `JLabel`. Contiene 10 `JTextField` per gli username dei giocatori.
- **Metodi Chiave:** `getGiocatoriUsernames()` (che raccoglie il testo dai 10 campi) e `getNomeSquadraA()`/`B()`.

10. `StoricoPrenotazioniDialog.java`

- **Ruolo Architetturale:** View (Finestra Pop-up).
- **Classe:** `JDialog`.
- **Descrizione Dettagliata:** Si apre quando il cliente clicca "Le mie Prenotazioni". Il costruttore riceve una `List<Prenotazione>` (fornita dal `PrenotazioneDAO` tramite il `AppController`) e la usa per riempire una `JTable`.

11. `TuttePrenotazioniDialog.java`

- **Ruolo Architetturale:** View (Finestra Pop-up).
- **Classe:** `JDialog`.
- **Descrizione Dettagliata:** Identica a `StoricoPrenotazioniDialog`, ma è per l'admin e mostra *tutte* le prenotazioni del sistema, non solo quelle di un utente.

📦 Il Modello (Package: `it.unipv.ingsfw.playfieldtech.model`)

Questi file sono le "entità" del programma. Sono classi Java semplici (POJO) che rappresentano i dati e mappano le tabelle del database.

12. `Utente.java`

- **Ruolo Architetturale:** Model (Entità Base).
- **Classe:** `abstract class`.
- **Descrizione Dettagliata:** È la classe genitore per `Cliente` e `Amministratore`. Definisce i campi comuni (id, username, password, nome, cognome). Essendo `abstract`, non può essere istanziata da sola.

13. `Cliente.java`

- **Ruolo Architetturale:** Model (Entità).
- **Classe:** `extends Utente`.
- **Descrizione Dettagliata:** Rappresenta un utente cliente. Aggiunge una `List<Prenotazione> storicoPrenotazioni` ai campi ereditati da `Utente`.

14. `Amministratore.java`

- **Ruolo Architetturale:** Model (Entità).
- **Classe:** `extends Utente`.
- **Descrizione Dettagliata:** Rappresenta un utente amministratore. Attualmente non aggiunge campi o metodi nuovi, ma la sua esistenza è fondamentale per la tipizzazione (permettere al Controller di fare `if (utenteCorrente instanceof Cliente)`).

15. `ImpiantoSportivo.java`

- **Ruolo Architetturale:** Model (Entità).
- **Descrizione Dettagliata:** Un semplice POJO che mappa la tabella `ImpiantiSportivi` (idCampo, nome, tipoSport).

16. `FasciaOraria.java`

- **Ruolo Architetturale:** Model (Entità).
- **Descrizione Dettagliata:** Mappa la tabella `FasceOrarie`. Utilizza i tipi moderni di Java `DayOfWeek` e `LocalTime` per gestire date e ore.

17. `Prenotazione.java`

- **Ruolo Architetturale:** Model (Entità "Aggregate").
- **Descrizione Dettagliata:** L'oggetto modello più complesso. Collega tutte le altre entità. Contiene un oggetto `Cliente` (chi prenota), un `ImpiantoSportivo` (dove), una `FasciaOraria` (quando) e una `List<Utente>` (chi gioca).
- **Campi Ausiliari:** `fkCampo` e `fkFascia`. Questi sono campi temporanei usati dal `PrenotazioneDAO` per costruire l'oggetto in modo efficiente: il DAO prima legge gli ID numerici dal DB, li mette in questi campi, e solo dopo il Controller (o un altro servizio) li "risolve" per caricare gli oggetti `ImpiantoSportivo` e `FasciaOraria` completi.

🗄️ Accesso ai Dati (Package: `it.unipv.ingsfw.playfieldtech.dao`)

Queste classi (DAO - Data Access Object) sono l'unico strato che può parlare direttamente con il database. Nascondono tutto l'SQL dal resto dell'app.

18. `UtenteDAO.java`

- **Ruolo Architetturale:** DAO.
- **Descrizione Dettagliata:** Gestisce la tabella `Utenti`.
- **Metodi Chiave:**
  - `validaLogin(user, pass)`: Esegue `SELECT * FROM Utenti WHERE username = ? AND password = ?`. Se trova un risultato, usa la `UtenteFactory` per costruire l'oggetto `Cliente` o `Amministratore`.
  - `salvaUtente(utente)`: Esegue `INSERT INTO Utenti...`. Usa `instanceof` per determinare se il `ruolo` da inserire è "cliente" o "amministratore".
  - `findUtenteByUsername(user)`: Esegue `SELECT * FROM Utenti WHERE username = ?`. Fondamentale per il Controller per validare i 10 giocatori.

19. `ImpiantoSportivoDAO.java`

- **Ruolo Architetturale:** DAO.

- **Descrizione Dettagliata:** Gestisce le tabelle `ImpiantiSportivi` e `Impianti_Fasce`.
- **Metodi Chiave:**
  - `getTuttiImpianti()`: Esegue `SELECT * FROM ImpiantiSportivi`.
  - `salvaImpiantoConFasce(impianto, fasce)`: Metodo **transazionale**. Usa `connection.setAutoCommit(false)` per assicurarsi che entrambe le operazioni (salvataggio impianto E salvataggio associazioni) vadano a buon fine. Se una fallisce, esegue `connection.rollback()` per annullare tutto, garantendo l'integrità dei dati.

20. `FasciaOrariaDAO.java`

- **Ruolo Architetturale:** DAO.
- **Descrizione Dettagliata:** Gestisce la tabella `FasceOrarie`.
- **Metodi Chiave:**
  - `getTutteFasceOrarie()`: Esegue `SELECT * FROM FasceOrarie`. Usa la clausola SQL `ORDER BY FIELD(giorno_settimana, ...)` per ordinare i giorni in modo logico (Lunedì, Martedì...) anziché alfabetico.

21. `ImpiantiFasceDAO.java`

- **Ruolo Architetturale:** DAO (per la tabella di collegamento).
- **Descrizione Dettagliata:** Metodo cruciale per le performance del calendario.
- **Metodi Chiave:**
  - `getMappaDisponibilita()`: Esegue `SELECT fk_campo, fk_fascia FROM Impianti_Fasce`. Invece di restituire una lista, costruisce una `Map<Integer, List<Integer>>` (una mappa "ID Campo" -> "Lista di ID Fasce"). Questo permette al Controller di verificare la disponibilità in memoria (molto velocemente) invece di interrogare il DB per ogni cella della tabella.

22. `PrenotazioneDAO.java`

- **Ruolo Architetturale:** DAO.
- **Descrizione Dettagliata:** Il DAO più complesso, gestisce `Prenotazioni` e `Prenotazioni_Giocatori`.
- **Metodi Chiave:**
  - `getPrenotazioniByDate(data)`: Esegue `SELECT * FROM Prenotazioni WHERE data_prenotazione = ?`. Usato dal calendario per trovare gli slot "OCCUPATO".
  - `salvaPrenotazione(prenotazione)`: Metodo **transazionale**. (1) Esegue `INSERT` in `Prenotazioni`, (2) recupera l'ID auto-generato (`RETURN_GENERATED_KEYS`), (3) Esegue un `INSERT` in *batch* (tutti insieme) per salvare i 10 giocatori in `Prenotazioni_Giocatori` usando il nuovo ID. Esegue `commit` o `rollback`.

    - `getTutteLePrenotazioniPerUtente(id)`: Usa una query `UNION` per combinare i risultati di due `SELECT`: (1) prenotazioni dove l'utente è `fk_cliente` e (2) prenotazioni dove l'utente è `fk_giocatore`. Usato per lo storico del cliente.
    - `getTutteLePrenotazioni()`: Esegue una `SELECT` con multipli `JOIN` (su Utenti, Impianti, Fasce) per dare all'admin una vista completa.

🏗️ Pattern Architetturali (Packages: `db`, `factory`, `manager`, `observer`)

Questi file sono "helper" che implementano specifici Design Pattern per mantenere il codice pulito, disaccoppiato e gestibile.

23. `DatabaseManager.java` (Package: `db`)

- **Ruolo Architetturale:** Singleton Pattern (Gestore Connessione).
- **Descrizione Dettagliata:** Classe fondamentale per l'accesso ai dati. Il suo costruttore è `private`. L'unico modo per ottenerne un'istanza è chiamare `DatabaseManager.getInstance()`.
- **Logica:** La prima volta che viene chiamato, crea la `Connection` JDBC (usando le credenziali hardcoded) e la salva. Tutte le chiamate successive restituiscono quella stessa connessione. Questo evita di aprire e chiudere connessioni al database continuamente, migliorando le performance. Tutti i DAO la usano.

24. `UtenteFactory.java` (Package: `factory`)

- **Ruolo Architetturale:** Factory Pattern.
- **Descrizione Dettagliata:** Ha un solo metodo, `creaUtente(tipo, ...)`.
- **Logica:** Viene chiamato dal `UtenteDAO` dopo una `SELECT`. Il DAO non sa quale oggetto creare; passa la stringa "ruolo" letta dal DB a questa factory, e la factory si occupa di fare `if ("cliente".equals...) return new Cliente(...)`. Questo disaccoppia il DAO (logica dati) dalla creazione specifica degli oggetti (logica business).

25. `Observer.java` (Package: `observer`)

- **Ruolo Architetturale:** Observer Pattern (Interfaccia Osservatore).
- **Descrizione Dettagliata:** Un'interfaccia con un solo metodo: `void update()`. Obbliga qualsiasi classe che la implementa (come `ClienteView`) a definire questo metodo, che verrà chiamato quando succede qualcosa.

26. `Subject.java` (Package: `observer`)

- **Ruolo Architetturale:** Observer Pattern (Interfaccia Soggetto).
- **Descrizione Dettagliata:** Un'interfaccia con i metodi `addObserver(o)`, `removeObserver(o)` e `notifyObservers()`. Obbliga qualsiasi classe che la implementa (come `PrenotazioneManager`) a gestire una lista di osservatori e a notificarli.

27. `PrenotazioneManager.java` (Package: `manager`)

- **Ruolo Architetturale:** Singleton Pattern e Subject Pattern.
- **Descrizione Dettagliata:** È sia un Singleton (`getInstance()`) per avere un unico punto di gestione, sia un `Subject`.
- **Logica:** Il `AppController` chiama `creaPrenotazione()`. Questo manager chiama il `prenotazioneDAO.salvaPrenotazione()`. Se il DAO restituisce `true` (salvataggio riuscito), il Manager chiama immediatamente `notifyObservers()`. Questo, a sua volta, esegue il metodo `update()` di tutti gli osservatori registrati (come `ClienteView`), che mostrerà il pop-up di conferma.